

Tyler Harris
599 Edge AI and Robotics
Module 1 Pred Classification

Overview:

This lab introduces the fundamentals of time-series forecasting using the PyTorch framework, optimized for Jetson devices. The module transitions from a “hand-coded” model to an automated training pipeline. By exploring weather data, solar power generation, and space shuttle telemetry, the lab demonstrates the application of RNNs and LSTM networks in both regression and classification. All completed notebooks are in the GitHub repo.

Setup:

This lab is run on a Docker container that has preconfigured PyTorch and CUDA libraries suited for the Jetson L4T.

Exercises:

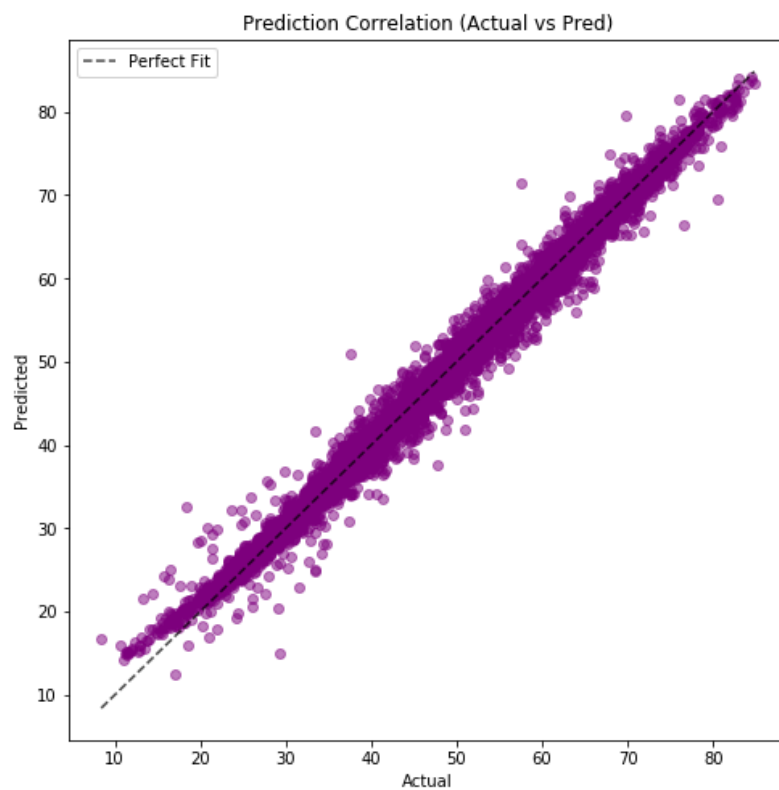
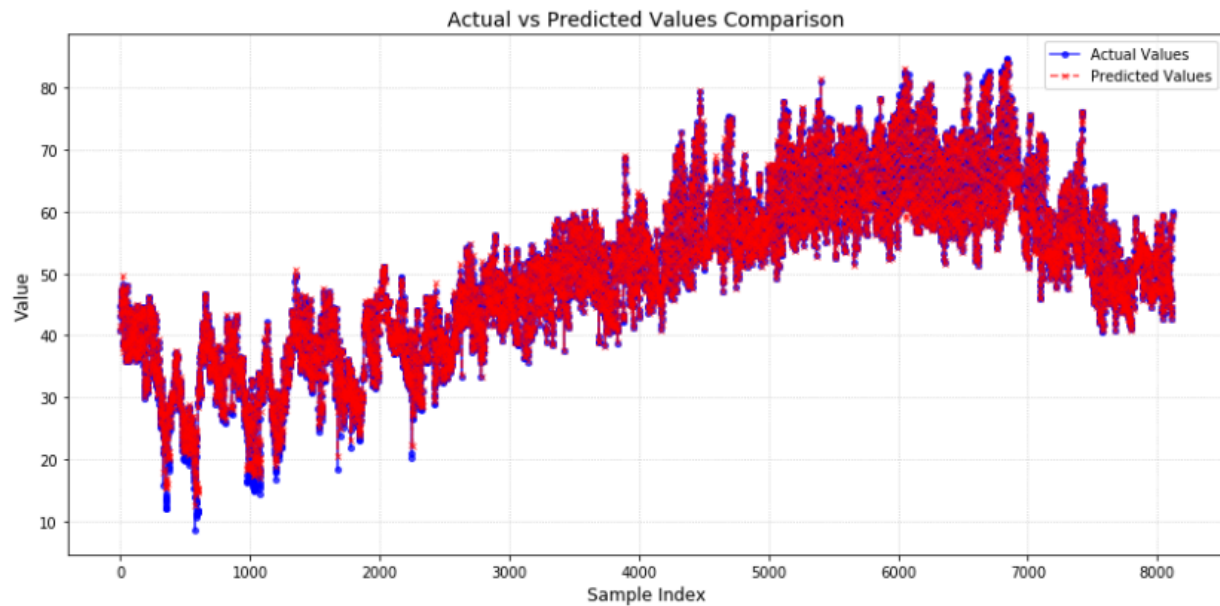
Notebook 1: Weather Forecasting (Hand Coded)

This notebook focuses on the ground up understanding of the PyTorch training loop. The goal is to predict temperature based on historical weather patterns. The main points are model architecture, loss function, and an optimizer (Adam).

Manual Training Setup (Weather Prediction):

- 1) Data is loaded from a csv into a pandas dataframe.
- 2) The temperature row is selected containing 40624 samples
- 3) Data is then preprocessed by applying scaler transformation normalizing temperature values to the range of $(-1, 1)$
- 4) PyTorch datasets are then created from the preprocessed data, this includes:
 - a) Transforming data from a numpy array to a tensor (and back if needed)
 - b) Ability to unscale the data
 - c) Generating sequences of a certain length since this is working with time-series data the sequence must be the proper length
 - d) A horizon is chosen to shift the labels and align them with the desired prediction
 - e) The sequence generation mentioned above is then used to build the sequences and the arrays are cast to tensors
 - f) The data is then split into training and testing lists
- 5) A model is then built, in this case a GRU
- 6) The forward propagation is defined
- 7) A loss function is defined, MSE for this example
- 8) The model is then trained for a specified number of epochs.

Results:



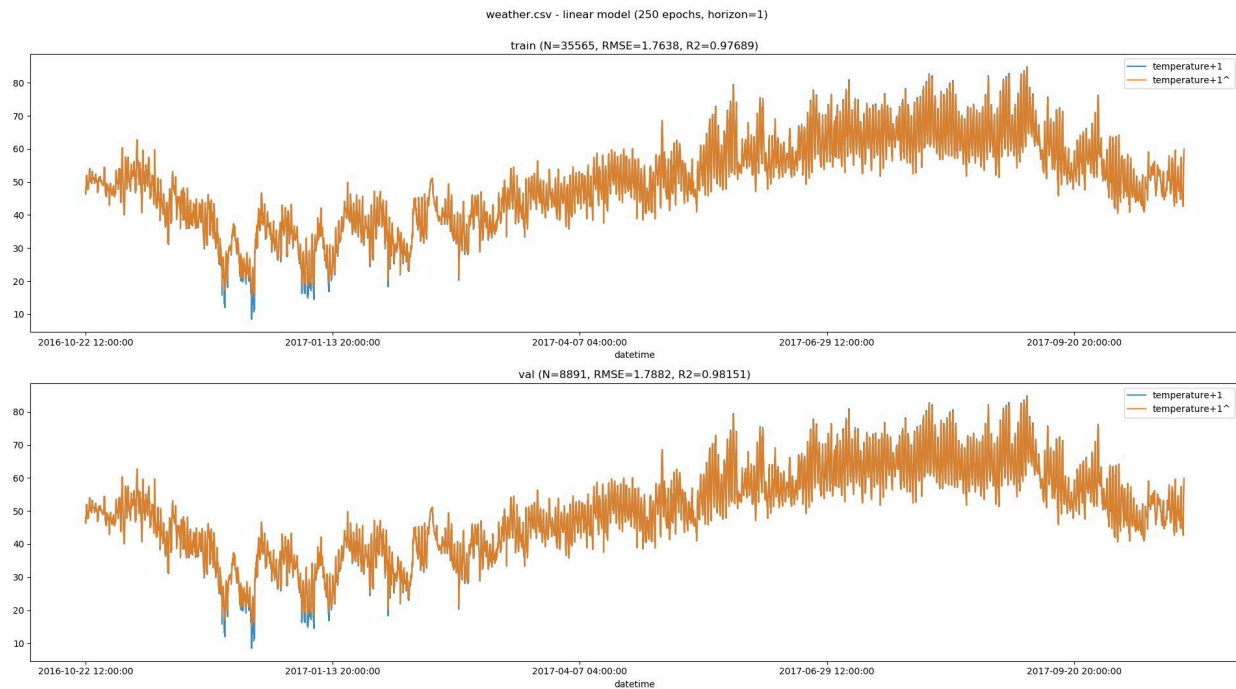
Observations:

The model performs relatively well with the predicted values clustering very closely to the line of best fit. There are instances where predicted values are much farther off however it performed well for a test example.

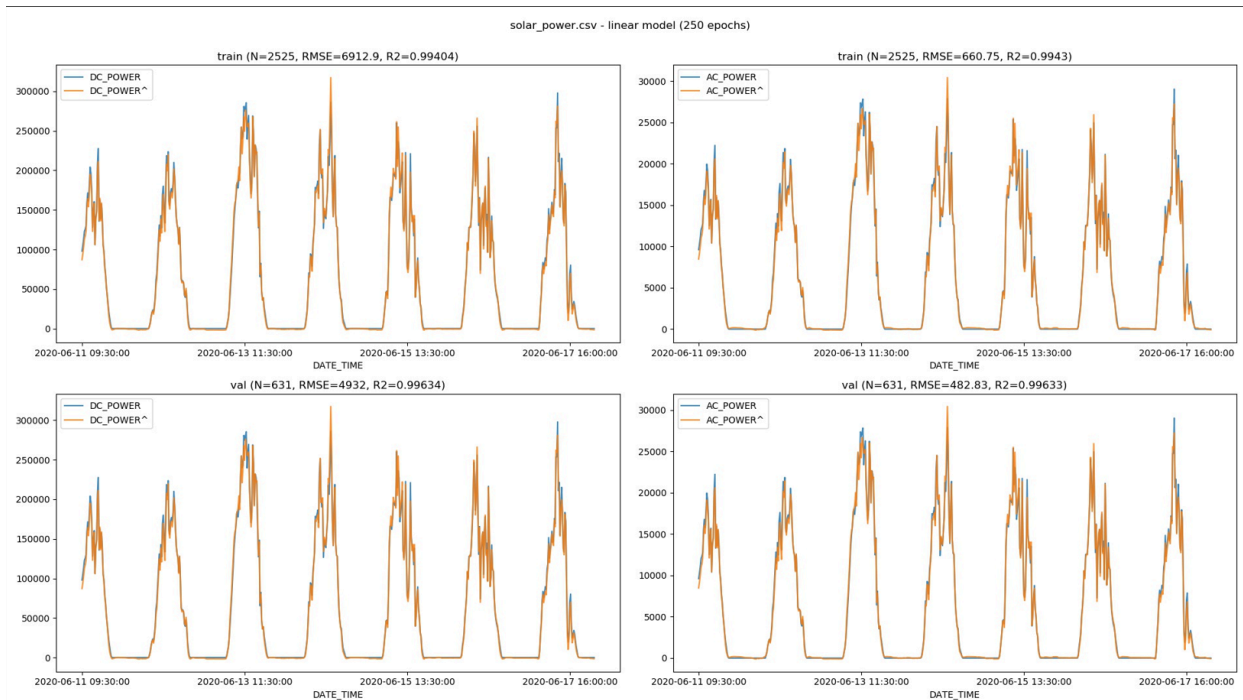
Subsequent notebooks abstract the entire training process, and instead are just given command line arguments as follows:

```
`python3 train.py -data <path to csv> -inputs <column selections> -outputs  
<output_parameters>`
```

Weather Prediction output (weather.csv):



Solar Power Prediction Output (solar_power.csv):



Space Shuttle Class Prediction (shuttle.csv):

best model: epoch=88 val_loss=0.03258512

train accuracy: [0.9993965517241379]
train precision: [0.9993870106753067]
train recall: [0.9993965517241379]
train F1: [0.9993778078357173]

/usr/local/lib/python3.6/dist-packages/sklearn/metrics/_classification.py:1248: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))
val accuracy: [0.9986206896551724]
val precision: [0.9982610992019746]
val recall: [0.9986206896551724]
val F1: [0.9984208423450618]

The last exercise was to supply a new dataset, I chose an electricity production data set (electric_production.csv):

